FIAP · DevOps Tools & Cloud Computing

Cidades ESG Inteligentes

Mobilidade Sustentável API — CI/CD, containerização e orquestração

Documentação técnica

Integrantes

RM	Nome
565788	Anna Clara Souza Andrade
564657	Cauã Rodrigo Sampaio
564772	Kelvin Wendell Barboza da Djalma Silva

Stack: Java 17 · Spring Boot 3.5 · Oracle 23 Free · Flyway · Docker · Docker Compose · GitHub Actions · GHCR

Data: outubro de 2026

Sumário

1. Visão geral do projeto
2. Pipeline CI/CD
3. Docker
4. Evidências do pipeline
5. Ambientes staging e produção funcionando
6. Desafios encontrados e soluções
7. Checklist de entrega

1. Visão geral do projeto

A aplicação escolhida é a API RESTful de Mobilidade Sustentável, desenvolvida em Java Spring Boot dentro do tema ESG. Empresas cadastram colaboradores, que registram deslocamentos sustentáveis (bicicleta, ônibus, carro elétrico). Cada deslocamento gera pontos proporcionais ao CO_2 evitado, que podem ser trocados por recompensas.

Para esta entrega a aplicação foi adaptada para simular um ambiente de produção real:

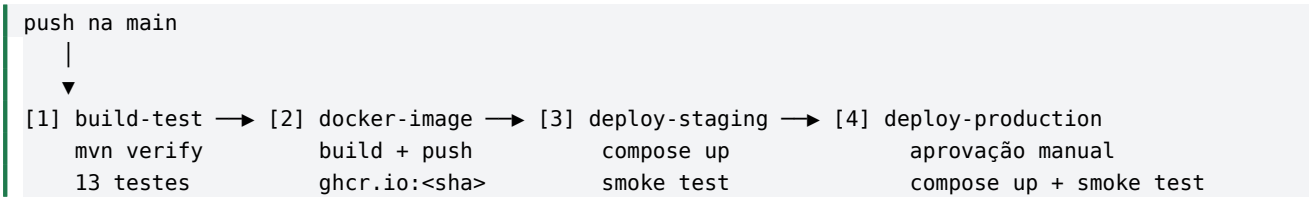
- **Configuração externa:** todas as senhas e URLs saíram do código e passaram a vir de variáveis de ambiente (.env local ou GitHub Secrets no pipeline).
- **Observabilidade:** Spring Boot Actuator com /actuator/health (usado nos health checks) e /actuator/info (mostra o ambiente e o commit em execução).
- **Documentação da API:** Swagger UI via springdoc-openapi.
- **Testes automatizados:** 13 testes (JUnit 5 + MockMvc + H2 em modo Oracle) cobrindo endpoints, validação e segurança.
- **Containerização:** Dockerfile multi-stage, usuário não-root, camadas do Spring Boot e HEALTHCHECK.
- **Orquestração:** Docker Compose com API + Oracle, redes separadas, volumes persistentes e arquivos específicos para staging e produção.
- **CI/CD:** GitHub Actions com build, testes, publicação da imagem e deploy automatizado em staging e produção.

2. Pipeline CI/CD

2.1 Ferramenta

GitHub Actions, definido em .github/workflows/ci-cd.yml. A imagem Docker é publicada no **GitHub Container Registry (ghcr.io)**. Os ambientes staging e production são **GitHub Environments**, cada um com seus próprios secrets e, em produção, aprovação manual.

2.2 Etapas



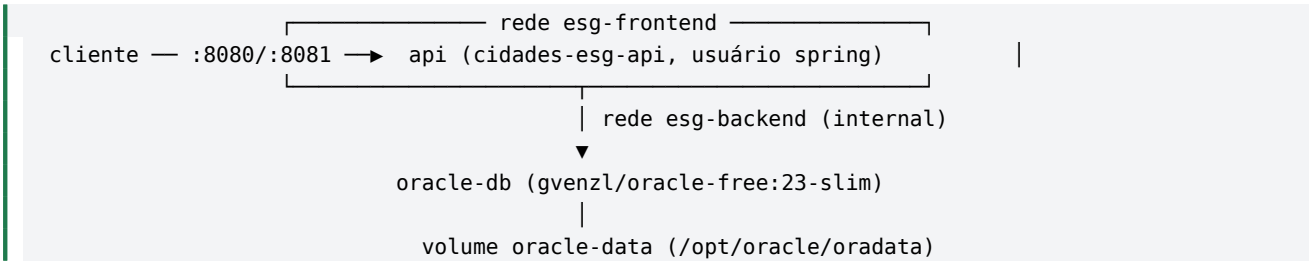
Etapa	Ação	Resultado
1. build-test	Checkout, JDK 17 (Temurin) com cache do Maven, mvn -B verify (compila, roda os testes e empacota).	Relatórios Surefire e app.jar como artefatos; resumo dos testes na página do job.
2. docker-image	Build multi-stage com Buildx e cache GHA; tags <sha do commit> e latest.	Imagem imutável em ghcr.io/<usuário>/<repo>:<sha>.
3. deploy-staging	Login no ghcr.io, gera .env.staging a partir dos secrets, scripts/deploy.sh staging (pull + up --wait), scripts/smoke-test.sh staging, scripts/evidencias.sh.	Staging em http://localhost:8081 validado; evidências no resumo do job e no artefato evidencias-staging.
4. deploy-production	Só roda se staging passou. Aguarda aprovação (Required reviewers). Mesmo deploy, smoke test e evidências.	Produção em http://localhost:8080 com a mesma imagem testada em staging.

2.3 Lógica e decisões

- **Gatilhos:** push na main executa o pipeline completo; pull request executa só CI (build, testes e build da imagem sem push); workflow_dispatch permite rodar manualmente.
- **Build once, deploy many:** a imagem é construída uma única vez e identificada pelo SHA do commit. Staging e produção recebem exatamente o mesmo artefato.
- **Gate de qualidade:** cada job depende do anterior (needs). Falha em teste impede a imagem; falha no smoke test de staging impede produção.
- **Aprovação manual:** o environment production exige revisão antes do deploy.
- **Segredos:** ORACLE_PASSWORD, DB_PASSWORD e API_ADMIN_PASSWORD ficam nos secrets de cada environment; o script de deploy grava o .env com permissão 600 no runner.
- **Smoke test pós-deploy:** verifica health, info (confere o nome do ambiente), GETs públicos, Swagger, 401 sem autenticação e um ciclo POST 201 / DELETE 204 com Basic Auth.
- **Runner de deploy configurável:** por padrão usa runner hospedado do GitHub (ambiente efêmero). Com a variável DEPLOY_RUNNER=self-hosted, o deploy roda num self-hosted runner com ambientes persistentes.
- **Concorrência:** cada ambiente tem um grupo de concurrency, evitando dois deploys simultâneos no mesmo ambiente.

3. Docker

3.1 Arquitetura



Recurso	Configuração
Serviço api	Imagem da aplicação; porta ${API_PORT}:8080; depende do banco healthy; volume app-logs; variáveis do .env.
Serviço oracle-db	gvenzl/oracle-free:23-slim; cria o usuário da aplicação no FREEPDB1; healthcheck.sh nativo.
Redes	esg-backend (internal: true — banco inacessível de fora); esg-frontend (só a API publica porta).
Volumes	oracle-data (persistência do banco) e app-logs (arquivo /app/logs/api.log).
Variáveis	Obrigatórias validadas com ${VAR:?mensagem}: ORACLE_PASSWORD, DB_PASSWORD, API_ADMIN_PASSWORD.
Local	docker-compose.override.yml: build da imagem e Oracle exposto só em 127.0.0.1:1521.
Staging	docker-compose.staging.yml: APP_ENV=staging, porta 8081, carga de dados de exemplo, limite de 768 MB.
Produção	docker-compose.prod.yml: APP_ENV=production, porta 8080, restart always, limite de 1 CPU / 1 GB, rotação de logs.

3.2 Dockerfile

```
# syntax=docker/dockerfile:1.7

# ---------- Etapa 1: build ----------
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /workspace

# Baixa dependencias primeiro (camada cacheada enquanto o pom.xml nao mudar)
COPY pom.xml .
RUN --mount=type=cache,target=/root/.m2 mvn -B -q dependency:go-offline

# Compila o codigo. Os testes rodam no pipeline (job "build-test"),
# por isso aqui sao pulados para nao duplicar tempo de build.
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -B -q -DskipTests package

# Separa o fat-jar em camadas (dependencias mudam pouco, codigo muda muito)
RUN java -Djarmode=tools -jar target/app.jar extract --layers --launcher --destination
/workspace/extracted

# ---------- Etapa 2: runtime ----------
FROM eclipse-temurin:17-jre-jammy AS runtime

LABEL org.opencontainers.image.title="cidades-esg-inteligentes-api" \
      org.opencontainers.image.description="API ESG de Mobilidade Sustentavel (Spring Boot +
Oracle)" \
      org.opencontainers.image.licenses="MIT"

# Usuario sem privilegios de root
RUN groupadd --system spring && useradd --system --gid spring --home /app spring \
    && mkdir -p /app/logs && chown -R spring:spring /app
WORKDIR /app

# Copia as camadas da menos para a mais volatil (melhor reaproveitamento de cache)
COPY --from=build --chown=spring:spring /workspace/extracted/dependencies/ ./
COPY --from=build --chown=spring:spring /workspace/extracted/spring-boot-loader/ ./
COPY --from=build --chown=spring:spring /workspace/extracted/snapshot-dependencies/ ./
COPY --from=build --chown=spring:spring /workspace/extracted/application/ ./

USER spring

ENV SERVER_PORT=8080 \
    APP_ENV=local \
    JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseContainerSupport
-Djava.security.egd=file:/dev/./urandom"

EXPOSE 8080

# Health check sem dependencias extras (curl/wget nao sao necessarios)
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
  CMD bash -c 'exec 3<>/dev/tcp/127.0.0.1/${SERVER_PORT} && printf "GET /actuator/health HTTP/1.0\r\
nHost: localhost\r\n\r\n" >&3 && grep -q "\"status\":\"UP\"" <&3' || exit 1

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS org.springframework.boot.loader.launch.JarLauncher"]
```

3.3 Estratégias

- **Multi-stage:** Maven e JDK só na etapa de build; a imagem final usa eclipse-temurin:17-jre-jammy.

- **Camadas do Spring Boot:** java -Djarmode=tools extract --layers separa dependências, loader e código. Alterações de código não invalidam a camada de dependências.
- **Cache:** dependency:go-offline antes de copiar o src e --mount=type=cache para o repositório Maven.
- **Segurança:** usuário não-root (spring) e nenhuma credencial na imagem.
- **Saúde:** HEALTHCHECK no /actuator/health usando /dev/tcp do bash (sem instalar curl). O Compose usa esse status no up --wait.
- **JVM ciente do container:** -XX:MaxRAMPercentage=75 respeita os limites de memória definidos no Compose.

3.4 Comandos usados

```
# Local
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f api
docker compose down            # -v apaga também o volume do banco

# Imagem
docker build -t cidades-esg-api:local .
docker run --rm -p 8080:8080 --env-file .env cidades-esg-api:local

# Ambientes (mesmos scripts usados pelo pipeline)
./scripts/deploy.sh staging     && ./scripts/smoke-test.sh staging
./scripts/deploy.sh production  && ./scripts/smoke-test.sh production

# Por baixo, o deploy executa:
docker compose --env-file .env.staging -f docker-compose.yml -f docker-compose.staging.yml config --
quiet
docker compose --env-file .env.staging -f docker-compose.yml -f docker-compose.staging.yml pull
docker compose --env-file .env.staging -f docker-compose.yml -f docker-compose.staging.yml up -d --
wait
```

3.5 Imagem criada

Item	Valor
Nome	ghcr.io/<usuário>/<repositório>
Tags	<SHA completo do commit> (imutável, usada no deploy) e latest
Base	eclipse-temurin:17-jre-jammy
Usuário	spring (não-root)
Porta	8080
Entrypoint	java $JAVA_OPTS org.springframework.boot.loader.launch.JarLauncher
Health check	GET /actuator/health a cada 15 s (start period 90 s)

4. Evidências do pipeline

Link da execução: `https://github.com/<usuário>/<repositório>/actions/runs/<id>`

[INSERIR PRINT 01-pipeline-visao-geral.png]

`docs/evidencias/01-pipeline-visao-geral.png`

GitHub → Actions → execução da main (grafo dos jobs)

Figura 1 — Execução completa do pipeline: 4 jobs concluídos

[INSERIR PRINT 02-build-testes.png]

`docs/evidencias/02-build-testes.png`

Job "Build e testes" → step "Compilar, testar e empacotar"

Figura 2 — Build e testes automatizados (Tests run: 13, Failures: 0)

[INSERIR PRINT 03-imagem-ghcr.png]

`docs/evidencias/03-imagem-ghcr.png`

Repositório → Packages → imagem com tag do commit

Figura 3 — Imagem publicada no GitHub Container Registry

[INSERIR PRINT 04-deploy-staging.png]

`docs/evidencias/04-deploy-staging.png`

Job "Deploy - Staging" → steps de deploy e smoke test (ou Summary)

Figura 4 — Deploy automatizado em staging com smoke test

[INSERIR PRINT 05-deploy-producao.png]

`docs/evidencias/05-deploy-producao.png`

Review deployments + job "Deploy - Producao"

Figura 5 — Aprovação e deploy em produção com smoke test

4.1 Validação estática dos arquivos de DevOps

Antes do primeiro push, o workflow, o Dockerfile, os scripts e os três arquivos Compose foram validados com ferramentas de lint. Saída real da execução:

```
$ date
Mon Oct 12 20:25:42 -03 2026

$ actionlint .github/workflows/ci-cd.yml
(sem erros)

$ hadolint --ignore DL3008 Dockerfile
(sem erros)

$ shellcheck scripts/*.sh
(sem erros)

$ docker compose --env-file .env.example config --quiet
(configuracao local valida)

$ docker compose -f docker-compose.yml -f docker-compose.staging.yml config --quiet
(configuracao staging valida)

$ docker compose -f docker-compose.yml -f docker-compose.prod.yml config --quiet
(configuracao producao valida)

$ javac (parse) src/**/*.java
26 arquivos Java sem erros de sintaxe
```

5. Ambientes staging e produção funcionando

[INSERIR PRINT 06-staging-swagger.png]

`docs/evidencias/06-staging-swagger.png`

http://localhost:8081/swagger-ui.html ou /actuator/info

Figura 6 — Staging (porta 8081): Swagger UI / actuator/info com "environment":"staging"

[INSERIR PRINT 07-producao-swagger.png]

`docs/evidencias/07-producao-swagger.png`

http://localhost:8080/swagger-ui.html ou /actuator/info

Figura 7 — Produção (porta 8080): Swagger UI / actuator/info com "environment":"production"

6. Desafios encontrados e soluções

Desafio	Solução
Senhas fixas no código (admin123 no SecurityConfig e senha do banco no application.properties).	Credenciais movidas para variáveis de ambiente (API_ADMIN_PASSWORD, DB_PASSWORD, ORACLE_PASSWORD), com .env.example versionado, .env no .gitignore e GitHub Secrets por environment.
Usuário do banco inconsistente entre README (APP_MOBILIDADE) e docker-compose (FIAP).	Uma única variável DB_USERNAME alimenta o Oracle (APP_USER) e a API.
O projeto não tinha testes automatizados.	Criados 13 testes de integração com MockMvc cobrindo GETs, 404, validação (400) e segurança (401/201).
As migrations Flyway usam SQL específico do Oracle, o que impede rodar os testes sem um Oracle no CI.	Perfil test com H2 em modo Oracle, Flyway desligado e schema gerado pelo Hibernate. O SQL real do Oracle é exercitado no deploy de staging.
O Oracle leva de 1 a 2 minutos para ficar pronto e a API falhava ao subir antes dele.	healthcheck.sh no banco + depends_on: service_healthy + docker compose up --wait + espera no smoke test.
A porta 1521 do banco ficava exposta.	Rede esg-backend com internal: true. A porta só é publicada no ambiente local e apenas em 127.0.0.1.
Staging e produção na mesma máquina colidiam (containers, volumes, portas).	Cada ambiente é um projeto Compose próprio (esg-staging / esg-production), com portas 8081/8080 e volumes separados.
A imagem JRE não tem curl para o HEALTHCHECK.	Health check feito com /dev/tcp do bash, sem aumentar a imagem.
Garantir que produção receba a mesma versão testada em staging.	Imagem imutável identificada pelo SHA do commit, promovida de staging para produção; produção só roda após staging verde e aprovação.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑
Dockerfile funcional	☑
docker-compose.yml ou arquivos Kubernetes	☑
Pipeline com etapas de build, teste e deploy	☑
README.md com instruções e prints	☐
Documentação técnica com evidências (PDF ou PPT)	☐
Deploy realizado nos ambientes staging e produção	☐